💾 Save It For Real

Persistent CRUD with SQLite

Session 3 of 3 · 90 minutes

The problem we left with 😱

Last week's tasks **disappeared** on restart.

> State lives in **memory** → memory clears when the app closes.

Today the app gets **permanent memory**: a real **database** that lives on the phone.

Today's goal 🎯

Upgrade the To-Do into a **real app**:

- 💾 tasks **survive** restarts (SQLite)
- 🔁 full **CRUD**: Create, Read, Update, Delete
- ✏️ an **edit screen** (a pop-up form)
- ✨ polish: confirm-delete, a floating ＋ button

Then: your **final project**. 🏁

Part 1

Where does data live?

Memory vs. storage

	State (memory)	Database (storage)
Survives restart?	❌ no	✅ yes
Speed	instant	very fast
Good for	what's on screen *now*	data you want to **keep**

We'll keep using state for the screen — and a **database** to remember.

Meet SQLite 

- A **real SQL database** that lives **inside your app**, on the phone
- No server, no internet, nothing to set up
- Comes with Expo: `expo-sqlite`
- The **same SQL** used by huge systems — just tiny & local

> Your data lives in a file on the device. It's still there tomorrow.

CRUD = the four things apps do

Letter	Action	SQL command
C	Create	`INSERT`
R	Read	`SELECT`
U	Update	`UPDATE`
D	Delete	`DELETE`

Almost every app is just CRUD on some data.

Part 2

Set up the database

Install & open

```
npx expo install expo-sqlite
```

```
import * as SQLite from 'expo-sqlite';

// open (or create) a database file on the device
const db = SQLite.openDatabaseSync('tasks.db');
```

`openDatabaseSync` gives us a `db` to run commands on.

Create a table (your data's shape)

```
db.execSync(`
  CREATE TABLE IF NOT EXISTS tasks (
    id    INTEGER PRIMARY KEY NOT NULL,
    title TEXT    NOT NULL,
    done  INTEGER NOT NULL DEFAULT 0
  );
`);
```

A **table** is like a spreadsheet: **columns** = fields, **rows** = items.

SQLite data types (the short list)

Type	Use for	Example
`INTEGER`	whole numbers, ids, true/false (0/1)	`id`, `done`
`TEXT`	any text	`title`, `note`
`REAL`	decimals	`price`

> No `BOOLEAN` and no `DATE` — use `INTEGER` (0/1) and `TEXT`.

Part 3

The four operations

➕ Create → `INSERT`

```
const result = db.runSync(
  'INSERT INTO tasks (title, done) VALUES (?, ?)',
  title, 0
);

result.lastInsertRowId;   // the new task's id
```

`runSync` runs a command that **changes** data.

📖 Read → `SELECT`

```
const tasks = db.getAllSync(
  'SELECT * FROM tasks ORDER BY id DESC'
);
// → [ { id: 3, title: '...', done: 0 }, ... ]
```

`getAllSync` returns an **array of rows** — exactly what `FlatList` wants!

✏️ Update → `UPDATE` · 🗑️ Delete → `DELETE`

```
// change a task's title
db.runSync('UPDATE tasks SET title = ? WHERE id = ?', title, id);

// flip done on/off (0 ↔️ 1)
db.runSync('UPDATE tasks SET done = ? WHERE id = ?', done ? 1 : 0, id);

// remove a task
db.runSync('DELETE FROM tasks WHERE id = ?', id);
```

> ⚠️ **Always include `WHERE id = ?`** — without it you change/delete **every** row!

Why the `?` placeholders matter

```
// ✅ safe — values go in the params
db.runSync('INSERT INTO tasks (title) VALUES (?)', title);

// ❌ never glue strings together
db.runSync(`INSERT INTO tasks (title) VALUES ('${title}')`);
```

Placeholders handle quotes for you and block **SQL injection**.

Part 4

Organize it: a `db.js` module

Keep database code in one file

```
// db.js
import * as SQLite from 'expo-sqlite';
const db = SQLite.openDatabaseSync('tasks.db');

export function setupDatabase() {
  db.execSync(`CREATE TABLE IF NOT EXISTS tasks (
    id INTEGER PRIMARY KEY NOT NULL,
    title TEXT NOT NULL, done INTEGER NOT NULL DEFAULT 0);`);
}
export function getTasks()        { return db.getAllSync('SELECT * FROM tasks ORDER BY id DESC'); }
export function addTask(title)    { db.runSync('INSERT INTO tasks (title, done) VALUES (?, ?)', title, 0); }
export function updateTask(id, t) { db.runSync('UPDATE tasks SET title = ? WHERE id = ?', t, id); }
export function toggleTask(id, d) { db.runSync('UPDATE tasks SET done = ? WHERE id = ?', d ? 1 : 0, id); }
export function deleteTask(id)    { db.runSync('DELETE FROM tasks WHERE id = ?', id); }
```

Part 5

Wire it into the screen

Load on startup with `useEffect`

```
import { useEffect, useState } from 'react';
import * as DB from './db';

const [tasks, setTasks] = useState([]);

useEffect(() => {
  DB.setupDatabase();   // create table (first run)
  refresh();            // load saved tasks
}, []);                 // [] = run ONCE when the app starts
```

The golden pattern: change → refresh

```
function refresh() {
  setTasks(DB.getTasks());     // re-read the DB into state
}

function add(title) {
  DB.addTask(title);           // 1) change the database
  refresh();                   // 2) re-read so the screen updates
}
```

> Database = the truth. State = a fresh copy for the screen.

Part 6

A second screen + polish

One `Modal`, two jobs (add and edit)

```
const [modalVisible, setModalVisible] = useState(false);
const [draft, setDraft] = useState('');
const [editingId, setEditingId] = useState(null); // null = adding

function save() {
  if (editingId === null) DB.addTask(draft);         // CREATE
  else                    DB.updateTask(editingId, draft); // UPDATE
  setModalVisible(false);
  refresh();
}
```

Confirm before deleting (`Alert`)

```
import { Alert } from 'react-native';

function remove(task) {
  Alert.alert('Delete task?', `"${task.title}"`, [
    { text: 'Cancel', style: 'cancel' },
    { text: 'Delete', style: 'destructive',
      onPress: () => { DB.deleteTask(task.id); refresh(); } },
  ]);
}
```

Never delete user data without asking. 🙏

"Real" navigation (for later) 🧭

A `Modal` is great for one pop-up. Multi-screen apps use a **navigation library**:

- **Expo Router** — files become screens (`app/index.js`, `app/details.js`)
- **React Navigation** — stacks & tabs

> Stretch goal / final project — not required today.

🎬 Live-code target

The finished **persistent To-Do**: list + floating ＋ → add/edit modal → tap to toggle → ✏️ edit → 🗑️ delete (with confirm) → **survives restart**.

🧪 Activity 3

Make your tasks permanent

Time: ~20 min · **Files:** `db.js` + `App.js`

1. Create **`db.js`** with `setupDatabase`, `getTasks`, `addTask`, `deleteTask`
2. In `App.js`, load tasks in **`useEffect`** (run once)
3. **Add** a task → save to DB → **refresh**
4. **Delete** a task → remove from DB → **refresh**
5. **Prove it:** force-close the app, reopen — tasks are still there ✅

Activity 3 — checkpoints & stretch

✅ **Done when:** tasks **survive a full restart**, and you can add + delete.

🌟 **Stretch goals:**

- **Toggle done** persists (`UPDATE ... SET done`)
- **Edit** a task with the Modal (`UPDATE ... SET title`)
- **Confirm delete** with `Alert`
- A **"saved 💾"** badge so users know it's permanent

🏁 Final Project

Build your own CRUD app

Your mission

Build a **SQLite-backed CRUD app** of your choice:

🗂️ Task manager · 💸 Expense tracker · 📝 Notes/Journal
📇 Contacts · 📦 Inventory · ✅ Habit tracker · 💡 *your idea*

Must have: a table (≥3 columns) · full **CRUD** · a list + an add/edit form · empty state · confirm-delete · clean styling.

📄 Full brief, milestones & rubric: `final-project/BRIEF.md`

You started at zero. Look now. 🚀

In three sessions you learned to:

- think in **components** & **state**
- handle **input**, **events**, and **lists**
- store data in a real **SQLite** database (full CRUD)
- ship a working app **on your own phone**

That's genuine mobile development. 👏

Where to go next 🧭

- 📚 **docs.expo.dev** — your reference
- 🧭 **Expo Router** — multi-screen apps
- 🎨 Component kits, icons, animations
- 🛠️ **Build the final project** — then keep building

Thank you! 🙌

Now go ship something.